Research Software Engineering in Python for Reproducible Computational Science

Sylwester Arabas

AGH University of Krakow

2026-03-06

plan for today

- RSE: what is it, or who is it?
- RSE with Python: why?
- scope of the course and the tech stack
- unit-aware coding in Python with Pint (code along!)
- practicalities next week!

plan for today

- **RSE: what is it, or who is it?**
- RSE with Python: why?
- scope of the course and the tech stack
- unit-aware coding in Python with Pint (code along!)
- practicalities next week!

RSE job title

LinkedIn
https://pl.linkedin.com › jobs › research-software-engine...

343 Research Software Engineer w Polska (17 nowych)

343oferty pracy – **Research Software Engineer** – Polska (nowe: 17) · Freelance Machine Learning Engineer (Python) · Junior Quantitative Developer · Machine Learning ...

Indeed
https://www.indeed.com › q-resear... • Tłumaczenie strony

Research Software Engineer Jobs, Employment

5632 **Research Software Engineer jobs** available on Indeed.com. Apply to Software Engineer, Research Engineer, Senior Software Engineer and more!

Society of Research Software Engineering
https://society-rse.org › careers • Tłumaczenie strony

Vacancies

RSE **Job Vacancies** · Newcastle University. Software Engineer/Full Stack Developer · University of Southampton. Senior **Research Software Engineer** · British Antarctic ...

Totaljobs
https://www.totaljobs.com › jobs • Tłumaczenie strony

Research Software Engineer job vacancies - Updated Daily

4,211 **Research Software Engineer jobs** ; Junior Embedded Software Research Engineer. Thales Group ; Research Software Engineer - Department of Applied Health ...

Google
https://careers.google.com › results • Tłumaczenie strony

Research Software Engineer, AI and Systems Research

Research Software Engineer, AI and Systems Research. share. linkCopy link ... Please do not forward resumes to our **jobs** alias, Google employees, or any other ...

AGH

RSE definitions

Gruenpeter et al. 2021 (doi:10.5281/zenodo.5504015)

Research Software *includes source code files, algorithms, scripts, computational workflows and executables that were* ***created during the research process or for a research purpose****.*

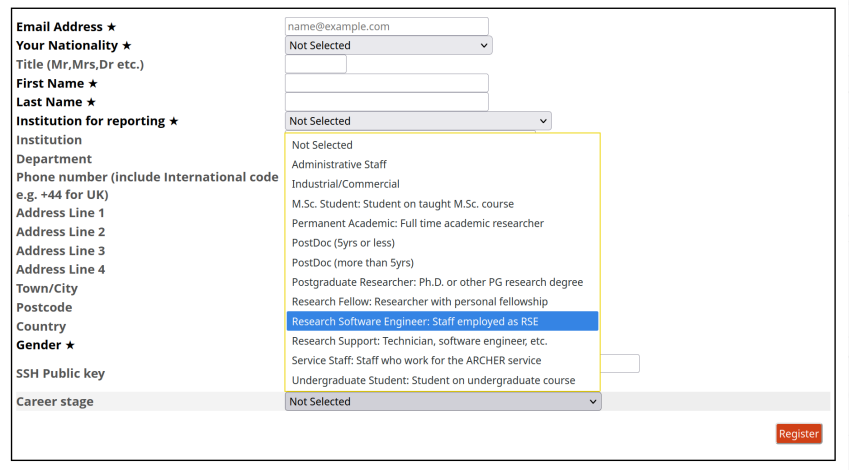
...minimal requirement for achieving ***computational reproducibility*** *is that all the computational components (Research Software, software used in research, documentation and hardware) used during the research are* **identified**, *described, and* ***made accessible*** *to the extent that is possible.*

https://society-rse.org/about/

A ***Research Software Engineer*** *(RSE) combines professional software engineering expertise with an intimate understanding of research.*

AGH

RSE as a career path



(https://safe.epcc.ed.ac.uk/signup.jsp)

RSE as a career path (doi:10.48550/arXiv.2602.19353)

arXiv:2602.19353 (cs)

[Submitted on 22 Feb 2026]

Designing and Implementing a Comprehensive Research Software Engineer Career Ladder: A Case Study from Princeton University

Ian A. Cosden, Elizabeth Holtz, Joel U. Bretheim

View PDF HTML (experimental)

Research Software Engineers (RSEs) have become indispensable to computational research and scholarship. The fast rise of RSEs in higher education and the trend of universities to be slow creating or adopting models for new technology roles means a lack of structured career pathways that recognize technical mastery, scholarly impact, and leadership growth. In response to an immense demand for RSEs at Princeton University, and dedicated funding to grow the RSE group at least two-fold, Princeton was forced to strategize how to cohesively define job descriptions to match the rapid hiring of RSE positions but with enough flexibility to recognize the unique nature of each individual position. This case study describes our design and implementation of a comprehensive RSE career ladder spanning Associate through Principal levels, with parallel team-lead and managerial tracks. We outline the guiding principles, competency framework, Human Resources (HR) alignment, and implementation process, including engagement with external consultants and mapping to a standard job leveling framework utilizing market benchmarks. We share early lessons learned and outcomes including improved hiring efficiency, clearer promotion pathways, and positive reception among staff.

Comments: Submitted to Future Generation Computer Systems: Special Issue on Research Software Engineering - Software-Enabled Discovery and Beyond

Subjects: **Software Engineering (cs.SE)**

Cite as: **arXiv:2602.19353 [cs.SE]**

my own RSE experience?

my own RSE experience?

- contributor to GNU Data Language since 2009 (mostly till 2021)

my own RSE experience?

- contributor to GNU Data Language since 2009 (mostly till 2021)
- 2 years of employment as a developer in financial sector (2015-2017)

my own RSE experience?

- contributor to GNU Data Language since 2009 (mostly till 2021)
- 2 years of employment as a developer in financial sector (2015-2017)
- open-source research-software developments while at UW, UJ, UIUC & AGH

my own RSE experience?

- contributor to GNU Data Language since 2009 (mostly till 2021)
- 2 years of employment as a developer in financial sector (2015-2017)
- open-source research-software developments while at UW, UJ, UIUC & AGH
- currently maintainer of PySDM, PyMPDATA, PyPartMC & Numba-MPI

AGH

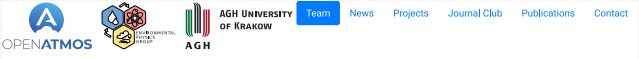
my own RSE experience?

- contributor to GNU Data Language since 2009 (mostly till 2021)
- 2 years of employment as a developer in financial sector (2015-2017)
- open-source research-software developments while at UW, UJ, UIUC & AGH
- currently maintainer of PySDM, PyMPDATA, PyPartMC & Numba-MPI
- editor at EGU Geoscientific Model Development journal since 2020

AGH

my own RSE experience?

- contributor to GNU Data Language since 2009 (mostly till 2021)
- 2 years of employment as a developer in financial sector (2015-2017)
- open-source research-software developments while at UW, UJ, UIUC & AGH
- currently maintainer of PySDM, PyMPDATA, PyPartMC & Numba-MPI
- editor at EGU Geoscientific Model Development journal since 2020
- spinning up RSE team at AGH: open-atmos-krk.github.io



We are a team of students, researchers and developers at the AGH University of Krakow in Poland. Our activities concern free/libre and open-source software for atmospheric sciences, and cloud physics in particular. We develop example-rich tools to support reproducible computational science, while promoting best practices in research software engineering. Our team contributes to the open-source ecosystem we rely on. We explore modern, transferable technologies that are useful both within and beyond academia. Most of the projects we develop, maintain, and contribute to are hosted at `github.com/open-atmos` (while the Kraków airport IATA code is KRK, hence `open-atmos-krk`). Before moving to the AGH Environmental Physics Group in 2023, we had been based at the Institute of Computer Science and Computational Mathematics, Jagiellonian University in Kraków.

plan for today

- RSE: what is it, or who is it?
- **RSE with Python: why?**
- scope of the course and the tech stack
- unit-aware coding in Python with Pint (code along!)
- practicalities next week!

Python's popularity

Programming, scripting, and markup languages

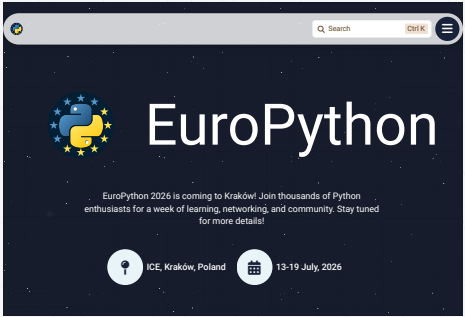
After more than a decade of steady growth, Python's adoption has accelerated significantly. It saw a 7 percentage point increase from 2024 to 2025; this speaks to its ability to be the go-to language for AI, data science, and back-end development.

? Which **programming, scripting, and markup languages** have you done extensive development work in over the past year, and which do you want to work in over the next year? (If you both worked with the language and want to continue to do so, please check both boxes in that row.)

https://survey.stackoverflow.co/2025/technology/

Python's popularity - Kraków is the place to be this summer!

https://europython.eu/



https://euroscipy.org/

Python's popularity - why?!

https://youtu.be/GfH4QL4VqJ0

Python as a glue language (⇝ performance essentially depends on what you glue together!)

The Wayback Machine - https://web.archive.org/web/19980614041513/http://www.objs.com:...

Glue It All Together With Python

Guido van Rossum
CNRI
1895 Preston White Drive
Reston, VA 20191
Email: guido@cnri.reston.va.us, guido@python.org

Position paper for the OMG-DARPA-MCC **Workshop on Compositional Software Architecture** in Monterey, California, January 6-8, 1998.

Introduction

Python is an advanced scripting language that is being used successfully to glue together large software components. It spans multiple platforms, middleware products, and application domains. Python is an object-oriented language with high-level data structures, dynamic typing, and dynamic binding. Python has been around since 1991, and has a very active user community. For more information, see the Python website http://www.python.org.

⇝ we got a language **without arrays**, where we **import** (JIT) compilers!

Python and computational science

Merali 2010: https://nature.com/articles/467775a

Python and computational science

Merali 2010: https://nature.com/articles/467775a

- *"...researchers do not test or document their programs rigorously, and they rarely release their codes, making it almost impossible to reproduce and verify published results ..."*

Python and computational science

Merali 2010: https://nature.com/articles/467775a

- *"...researchers do not test or document their programs rigorously, and they rarely release their codes, making it almost impossible to reproduce and verify published results ..."*
- *"...wrestling matches with wonky computer software ..."*

Python and computational science

Merali 2010: https://nature.com/articles/467775a

- *"...researchers do not test or document their programs rigorously, and they rarely release their codes, making it almost impossible to reproduce and verify published results ..."*
- *"...wrestling matches with wonky computer software ..."*
- *"...rewrote the code in a newer, more transparent programming language — Python — reducing its length and making it easier for people who aren't software experts to understand how it functions."*

Python and computational science

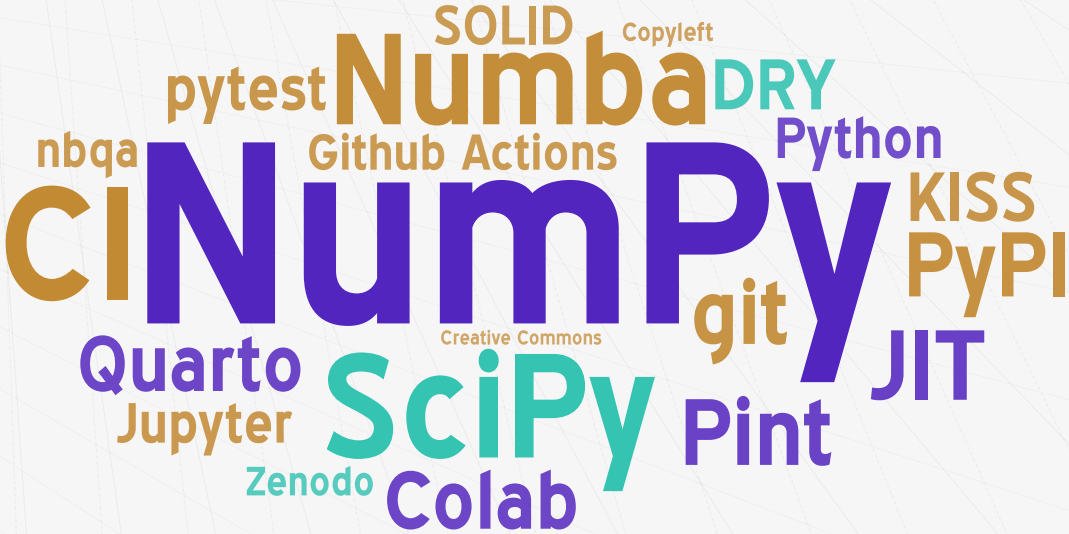
Merali 2010: https://nature.com/articles/467775a

- *"...researchers do not test or document their programs rigorously, and they rarely release their codes, making it almost impossible to reproduce and verify published results ..."*
- *"...wrestling matches with wonky computer software ..."*
- *"...rewrote the code in a newer, more transparent programming language — Python — reducing its length and making it easier for people who aren't software experts to understand how it functions."*
- *"...there needs to be a change in the way that science students are trained."*

plan for today

- RSE: what is it, or who is it?
- RSE with Python: why?
- **scope of the course and the tech stack**
- unit-aware coding in Python with Pint (code along!)
- practicalities next week!

Python packages, services and concepts for this course



Python in the cloud (colab.google, mybinder.org, ...)

$\rightsquigarrow$ mybinder.org

key goal of this course

develop/refine practical Python skills for conducting a reproducible BEng/MSc research project

plan for today

- RSE: what is it, or who is it?
- RSE with Python: why?
- scope of the course and the tech stack
- **unit-aware coding in Python with Pint (code along!)**
- practicalities next week!

Pint: docs and installation (try on mybinder.org or Colab)

https://pint.readthedocs.io/

```
!pip install --quiet pint
import pint
```

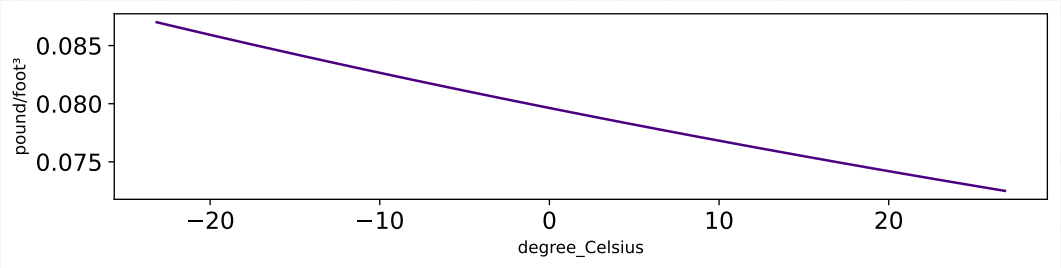
unit conversions & unit-aware coding

```python
import pint

si = pint.UnitRegistry()

R = 287 * si.J / si.K / si.kg
ρ = lambda T, p: p / (R * T)

print(f"{ρ(300 * si.K, 1000 * si.hPa).to_base_units()=:.2~}")
```

```
ρ(300 * si.K, 1000 * si.hPa).to_base_units()=1.2 kg / m ** 3
```

plotting

```
import numpy as np
from matplotlib import pyplot

si.setup_matplotlib()
pyplot.plot(T := np.linspace(250, 300) * si.K, ρ(T, p=1000*si.hPa))
pyplot.gca().xaxis.set_units(si.degC)
pyplot.gca().yaxis.set_units(si.lb / si.ft**3)
```



dimensional analysis and testing

```
try:
    R + R**2
except Exception as e:
    print(e)
```

```
Cannot convert from 'joule / kelvin / kilogram' ([length] ** 2 /
[time] ** 2 / [temperature]) to 'joule ** 2 / kelvin ** 2 /
kilogram ** 2' ([length] ** 4 / [time] ** 4 / [temperature] ** 2)
```

```
print(T.check('[temperature]'))
```

```
True
```

Pint: summary

- helps with (making code shorter, more readable, maintainable, and less bug prone):

Pint: summary

- helps with (making code shorter, more readable, maintainable, and less bug prone):
  - avoiding using comments for units in favour of programatic expressions

Pint: summary

- helps with (making code shorter, more readable, maintainable, and less bug prone):
  - avoiding using comments for units in favour of programatic expressions
  - handling unit conversions

Pint: summary

- helps with (making code shorter, more readable, maintainable, and less bug prone):
  - avoiding using comments for units in favour of programatic expressions
  - handling unit conversions
  - raising exceptions for unphysical arithmetics or function calls

Pint: summary

- helps with (making code shorter, more readable, maintainable, and less bug prone):
  - avoiding using comments for units in favour of programatic expressions
  - handling unit conversions
  - raising exceptions for unphysical arithmetics or function calls
  - physics-driven testing (i.e. dimensional analysis of the code)

Pint: summary

- helps with (making code shorter, more readable, maintainable, and less bug prone):
  - avoiding using comments for units in favour of programatic expressions
  - handling unit conversions
  - raising exceptions for unphysical arithmetics or function calls
  - physics-driven testing (i.e. dimensional analysis of the code)
  - automatic labelling and consistency checks for plot axes

AGH

Pint: summary

- helps with (making code shorter, more readable, maintainable, and less bug prone):
  - avoiding using comments for units in favour of programatic expressions
  - handling unit conversions
  - raising exceptions for unphysical arithmetics or function calls
  - physics-driven testing (i.e. dimensional analysis of the code)
  - automatic labelling and consistency checks for plot axes
  - last but not least: debugging!

AGH

Pint: summary

- helps with (making code shorter, more readable, maintainable, and less bug prone):
  - avoiding using comments for units in favour of programatic expressions
  - handling unit conversions
  - raising exceptions for unphysical arithmetics or function calls
  - physics-driven testing (i.e. dimensional analysis of the code)
  - automatic labelling and consistency checks for plot axes
  - last but not least: debugging!
- alternatives:

Pint: summary

- helps with (making code shorter, more readable, maintainable, and less bug prone):
  - avoiding using comments for units in favour of programatic expressions
  - handling unit conversions
  - raising exceptions for unphysical arithmetics or function calls
  - physics-driven testing (i.e. dimensional analysis of the code)
  - automatic labelling and consistency checks for plot axes
  - last but not least: debugging!
- alternatives:
  - C++: Boost.Units

AGH

Pint: summary

- helps with (making code shorter, more readable, maintainable, and less bug prone):
  - avoiding using comments for units in favour of programatic expressions
  - handling unit conversions
  - raising exceptions for unphysical arithmetics or function calls
  - physics-driven testing (i.e. dimensional analysis of the code)
  - automatic labelling and consistency checks for plot axes
  - last but not least: debugging!
- alternatives:
  - C++: Boost.Units
  - Julia: Unitful.jl

AGH

Pint: summary

- helps with (making code shorter, more readable, maintainable, and less bug prone):
  - avoiding using comments for units in favour of programatic expressions
  - handling unit conversions
  - raising exceptions for unphysical arithmetics or function calls
  - physics-driven testing (i.e. dimensional analysis of the code)
  - automatic labelling and consistency checks for plot axes
  - last but not least: debugging!
- alternatives:
  - C++: Boost.Units
  - Julia: Unitful.jl
  - JAX: unxt

plan for today

- RSE: what is it, or who is it?
- RSE with Python: why?
- scope of the course and the tech stack
- unit-aware coding in Python with Pint (code along!)
- **practicalities next week!**

Pythonicity and minimalism

```python
import this
```

```
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
```

Thank you for your attention!

slides and other course resources at
https://github.com/open-atmos-krk/rse-python-course